

(P) ~~FF~~FCFS. Write a program to implement FCFS

```
#include <stdio.h>

int main () {
    int n, i;
    int at[20], bt[20], start[20], tat[20], rt[20],
    float avgwt = 0, avgat = 0, avgt = 0;
    printf ("enter No. ");
    scanf ("%d", &n);
    printf ("enter Arrival Time: \n");
    for (i=0; i<n; i++) {
        printf ("P %d:", i+1);
        scanf ("%d", &at[i]);
        printf ("BT: ");
        for (i=0; i<n; i++) {
            printf ("P %d:", i+1);
            scanf ("%d", &bt[i]);
        }
        start[0] = at[0];
        ct[0] = start[0] + bt[0];
        for (i=1; i<n; i++) {
            if (ct[i-1] > at[i])
                start[i] = ct[i-1];
            else
                start[i] = at[i];
            ct[i] = start[i] + bt[i];
        }
        for (i=0; i<n; i++) {
            tat[i] = ct[i] - at[i];
            rt[i] = ct[i] - at[i];
        }
    }
}
```

3

```
for (i=0; i<n; i++) {
```

3

returns 0;

2

Enter number of processes: 2

PI AT: 1

$P_1 \quad B_1 : 2$

12 AF : 3

P2 BT : 4

Protein	AT	BT	CT	TAT	WT	RT
PI	1	2	3	4	5	6

(c) Non Preemptive STF

```
int main() {
```

first argst = 0, argtat = 0; argst = 0;

```
printf("Enter number of processes: ");  
scanf("%d", &n);
```

printing Center Arrival Time and Burst Time: (n)

```
for (i=0; i<n; i++) {
    printf("P%d AT: ", i+1);
    scanf("%d", &arr[i]);
}
```

```
printf ("P%d BT: ", i+1);  
scanf ("%d", &bt[i]);
```

9

while (Completed < n) {

int shortest = -1;

int minbt = INT_MAX;

for (i = 0; i < n; i++) {

if (at[i] <= time && done == 0 && bt[i] < minbt) {

minbt = bt[i];

shortest = i;

}

if (shortest == -1) {

time++;

continue;

}

at[shortest] = time - at[shortest];

time += bt[shortest];

ct[shortest] = time;

rt[shortest] = ct[shortest] - at[shortest];

wt[shortest] = rt[shortest] - bt[shortest];

done[shortest] = 1;

completed++;

avgwt += wt[shortest];

avgrt += rt[shortest];

avgct += ct[shortest];

}

printf("\n Process AT BT CT TAT WT RT\n");

for (i = 0; i < n; i++) {

printf("P %d %d %d %d %d %d %d\n", i+1, at[i], bt[i], ct[i], rt[i], wt[i], rt[i]);

}

printf("\n Average waiting Time = %.2f", avgwt/n);

printf("\n Average Turnaround Time = %.2f", avgrt/n);

printf("\n Average Response Time = %.2f", avgct/n);

return 0;

}

output >>

Enter number of processes: 2

Enter arrival time and burst time:

P1 AT: 12

P1 BT: 3h

P2 AT: 56

P2 BT: 67

Process	AT	BT	CT	TAT	WT	RT
P1	12	3h	46	3h	0	0
P2	56	67	123	67	0	0

waiting Time = 0.00

Average Turnaround Time = 50.50

Average Response Time = 0.00

Q. Greedy SJF

```
#include <stdio.h>
#include <limits.h>
```

```
int main() {
```

```
int n, i, time = 0, completed = 0;
int at[20], bt[20], rembt[20];
int ct[20], rt[20], wt[20], st[20];
int first[20] = {0};
```

```
float avgwt = 0, avgst = 0, avgtat = 0;
```

```
printf("Enter number of processes: ");
scanf("%d", &n);
```

```
printf("Enter Arrival and Burst Time: \n");
```

```
for (i = 0; i < n; i++) {
    printf("P%d AT: ", i+1);
    scanf("%d", &at[i]);
```

```
    printf("P%d BT: ", i+1);
    scanf("%d", &bt[i]);
```

```
    rembt[i] = bt[i];
    st[i] = -1;
```

```
}
```

```
while (completed < n) {
```

```
    int shortest = -1;
    int minbt = INT_MAX;
```

```
    for (i = 0; i < n; i++) {
```

```
        if (at[i] <= time && rembt[i] > 0 && rembt[i] < minbt) {
```

```
            minbt = rembt[i];
            shortest = i;
```

```
        }
```

```
    }
```

```
    if (shortest == -1) {
```

```
        rt[shortest] = time - at[shortest];
```

```
        rembt[shortest]--;
        time++;
```

```
    if (rembt[shortest] == 0) {
```

```
        completed++;
        ct[shortest] = time;
```

```
        tat[shortest] = ct[shortest] - at[shortest];
        wt[shortest] = tat[shortest] - bt[shortest];
```

```
        avgwt += wt[shortest];
        avgtat += tat[shortest];
        avgst += st[shortest];
```

```
    }
```

```
}
```

```
printf("In Process AT BT CT AT WT RT \n");
```

```
for(i=0; i<n; i++) {
```

```
printf("P%d \t %d \t %d \t %d \t %d \t %d \t %d \n",
```

```
i+1, at[i], bt[i], ct[i], tat[i], wt[i], rt[i]);
```

```
}
```

```
printf("\n Average Waiting Time = %.2f", avgwt);
```

```
printf("\n Average Turnaround Time = %.2f", avgat);
```

```
printf("\n Average Response Time = %.2f", avgrt);
```

```
return 0;
```

```
}
```

output>>

Enter number of processes: 2

Enter Arrival Time and Burst Time:

P1 AT: 12

P2 BT: 56

P2 AT: 18

P2 BT: 79

Process	AT	BT	CT	TAT	WT	RT
P1	12	56	68	56	0	0
P2	18	79	147	129	50	50

Average Waiting Time = 25.00

Average Turnaround Time = 92.50

Average Response Time = 25.00

LAB-2

(Q) Priority (Preemptive)

```
#include <stdio.h>
```

```
#include <limits.h>
```

```
int main() {
```

```
int n, i, time = 0, completed = 0;
```

```
int at[20], bt[20], pr[20], rembt[20];
```

```
int ct[20], tat[20], wt[20], rt[20];
```

```
float avgwt = 0, avgat = 0, avgrt = 0;
```

```
printf("Enter number of processes: ");
```

```
scanf("%d", &n);
```

```
printf("Enter Arrival Time, Burst Time and Priority:");
```

```
for(i=0; i<n; i++) {
```

```
printf("P%d AT: ", i+1);
```

```
scanf("%d", &at[i]);
```

```
printf("P%d BT: ", i+1);
```

```
scanf("%d", &bt[i]);
```

```
printf("P%d Priority: ", i+1);
```

```
scanf("%d", &pr[i]);
```

```
rembt[i] = bt[i];
```

```
rt[i] = -1;
```

```
}
```

```
while ( completed < n ) {
```

At highest = -1;

```
int min = INT_MAX;
```

```
for (i=0; i<n; i++) {
```

if (cat[i] <= time & number[i] > 0 & prob[i] < minProb)

```
min pr = pr[0];
highest = 1;
```

if (highest == -1) {

Time + + :

continuous;

```
if (int Onighest == -1) {
```

$$-at[\text{highest}] = \text{time} - at[\text{highest}];$$

```
rembet[highest]--;
```

time $t + t'$

if (rember [highest] == 0) {

Completed + +

at [highest] = time;

$$\text{ket}[\text{highest}] = \text{et}[\text{highest}] - \text{ar}[\text{highest}]$$
$$wt[\text{highest}] = \text{tot}[\text{highest}] - kt[\text{highest}]$$

```
mid = left + (right - left) / 2;
```

```
avgVal += bar[highest];
```

right += str(highest);

2

primary C¹⁰ in presence of BT OR CT TAT WT R¹⁰ 11

```
for (i=0; i<n; i++) {
```

```

    if (i == 0) {
        printf("1 1 1 1 1 1 1 1 1 1\n");
        return;
    }
    int a[i], b[i], p[i], c[i], k[i],
        w[i], r[i];

```

printf ("Average waiting Time = %.2f", avg_wait);

Print 1 "In Average Time measured Time = 4.21", (right);

print(L" in Average Response Time = 1.2710, right);

gablen 0;

Output $\Rightarrow$ Enter Number of processes: 2

Enter Arrival Time, Burst Time and Priority:

PI AT, 0

~~P1-BT: 4~~

P1 Priority: 2

P2 AT: 1

P2 AT: S

P2 Priority: 1

AT BT PR LT TAF LOT RT

P1 0 4 2 8 8 4 0

12 1 3 1 4 3 0 0

Average Waiting Time = 2.00

Average Turbidity Time = 5.50

Q1) Priority (Non Preemptive)

```
#include <stdio.h>
#include <limits.h>
```

```
int main() {
```

```
    int n, i, time = 0, completed = 0;
    int at[20], bt[20], pr[20];
    int ct[20], tat[20], wt[20], rt[20];
    int done[20] = {0};
```

```
    float avgat = 0, avgtat = 0, avgrt = 0;
    printf("Enter number of processes: ");
    scanf("%d", &n);
```

```
    printf("Enter Arrival Time, Burst Time and Priority");
```

```
    for (i = 0; i < n; i++) {
```

```
        printf("P %d AT: ", i+1);
        scanf("%d", &at[i]);
```

```
        printf("P %d BT: ", i+1);
        scanf("%d", &bt[i]);
```

```
        printf("P %d Priority: ", i+1);
        scanf("%d", &pr[i]);
    }
```

```
while (completed < n) {
```

```
    int highest = -1;
    int minpr = INT_MAX;
```

```
    for (i = 0; i < n; i++) {
        if (at[i] <= time && done[i] == 0 && pr[i] <= minpr) {
```

```
            minpr = pr[i];
            highest = i;
```

```
        }
```

```
    if (highest == -1) {
        time++;
        continue;
```

```
    rt[highest] = time - at[highest];
```

```
    time += bt[highest];
    ct[highest] = time;
```

```
    tat[highest] = ct[highest] - at[highest];
    wt[highest] = tat[highest] - bt[highest];
```

```
    avgrt += wt[highest];
    avgtat += tat[highest];
    avgat += rt[highest];
```

```
    done[highest] = 1;
    completed++;
```

```
}
```

```
printf("\n Process AT BT PR CT TAT WTRT\n");
```

```
for (i=0; i<n; i++) {
    printf("P%d \td \td \td \td \td \td\n",
           i+1, at[i], bt[i], pr[i], ct[i],
           wt[i], rt[i]);
}
```

```
printf("Average Waiting Time = %.2f", avgwt);
printf("Average Turnaround Time = %.2f", avgat);
printf("Average Response Time = %.2f", avrt);
```

```
return 0;
```

Output >> Enter number of processes: 2

P1 AT: 0

P1 BT: 4

P1 Priority: 2

P2 AT: 1

P2 BT: 3

P2 Priority: 1

Process	AT	BT	PR	CT	TAT	WTRT
P1	0	4	2	4	4	0
P2	1	3	1	7	6	3

Average Waiting Time = 1.50

Average Turnaround Time = 5.00

Average Response Time = 1.50

LAB-3

* Reason on Preemptive priority P2 interrupts P1 at time 7 because

Round Robin and Multilevel Queue

(0) Round Robin (different Quantization time)

#include <stdio.h>

```
int main() {
```

```
    int n, tq;
```

```
    int at[20], bt[20], rtRemaining[20], ct[20],
        tat[20], wt[20], rt[20], firstResponse[20];
```

```
    int queue[1000], front=0, rear=0;
```

```
    int inQueue[20] = {0};
```

```
    int i, time=0, completedCount=0;
```

```
    float avwt=0, avtat=0, avrt=0;
```

```
    printf("Round Robin (RR) CPU scheduling\n");
```

```
    printf("Enter total number of processes: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter the time Quantum: ");
```

```
    scanf("%d", &tq);
```

```
    for (i=0; i<n; i++) {
```

```
        printf("Process P[%d]\n", i+1);
```

```
        printf("Arrival Time: ");
```

```
        scanf("%d", &at[i]);
```

```
        printf("Burst Time: ");
```

```
        scanf("%d", &bt[i]);
```

```

    atRemaining[i] = bt[i];
    firstResponse[i] = -1;
}

```

```

int min_at = 999;
for (i=0; i<n; i++) {
    if (at[i] < min_at) {
        min_at = at[i];
    }
}
time = min_at;

```

```

for (i=0; i<n; i++) {
    if (at[i] <= time && !inQueue[i]) {
        queue[rear++] = i;
        inQueue[i] = 1;
    }
}

```

```

while (completedCount < n) {
    if (front == rear) {
        time++;
        for (i=0; i<n; i++) {
            if (at[i] <= time && !inQueue[i]) {
                queue[rear] = i;
                inQueue[i] = 1;
            }
        }
        continue;
    }
}

```

```

int idn = queue[front++];

```

```

if (firstResponse[idn] == -1) {
    firstResponse[idn] = time;
    rt[idn] = firstResponse[idn] - at[idn];
}

```

```

int execTime = (atRemaining[idn] > tq) ? tq :
    atRemaining[idn];
atRemaining[idn] -= execTime;
time += execTime;

```

```

for (i=0; i<n; i++) {
    if (at[i] <= time && !inQueue[i] &&
        atRemaining[i] > 0) {
        queue[rear++] = i;
        inQueue[i] = 1;
    }
}

```

```

if (atRemaining[idn] > 0) {
    queue[rear++] = idn;
} else {
    ct[idn] = time;
    ft[idn] = ct[idn] - at[idn];
    wt[idn] = ft[idn] - bt[idn];
    completedCount++;
}
}

```

```

printf("\n Process \t AT \t BT \t CT \t TAT \t RT\n");
for(i=0; i<n; i++) {
    avwt += wt[i];
    avtat += tat[i];
    avrt += rt[i];
    printf("\nP[%d] \t %d \t %d \t %d \t %d \t %d\n",
           i, at[i], bt[i], ct[i], tat[i], rt[i]);
}

```

```

printf("\n\nTime Quantum used: %d", tq);
printf("\n Average Waiting Time: %.2f", avwt/n);
printf("\n Average Turnaround Time: %.2f", avtat/n);
printf("\n Average Response Time: %.2f", avrt/n);

```

return 0;

}

output >> Round Robin (RR) CPU scheduling

Enter total number of processes: 3

Enter time Quantum: 3

Process P[1]

Arrival time: 2

Burst time: 6

Process P[2]

Arrival time: 4

Burst time: 8

Process P[3]

Arrival time: 6

Burst time: 7

Process	AT	BT	CT	TAT	WT	RT
P[1]	2	6	11	9	3	0
P[2]	4	8	22	18	10	1
P[3]	6	7	23	17	10	5

Time Quantum used: 3

Average waiting time: 7.67

Average Turnaround time: 14.67

Average Response time: 2.00

output >> Enter total number of processes 3
Enter time Quantum: 5

Process P[1]

Arrival time: 2

Burst time: 6

Process P[2]

Arrival time: 4

Burst time: 8

Process P[3]

Arrival time: 6

Burst time: 7

Process	AT	BT	CT	TAT	WT	RT
P[1]	2	6	18	16	10	0
P[2]	4	8	22	17	9	3
P[3]	6	7	23	17	10	6

Time Quantum used: 5

Average Waiting time: 9.67

Average Turnaround time: 16.67

Average Response time: 3.00

Comparison →

When the time quantum is less, it has a better response time but has lower CPU efficiency and for higher time quantum, response time is poor but CPU efficiency is higher.

Gantt Chart

Label	P1	P2	P3	P1	P2	P3	P2	P3
state	0	2	5	8	11	14	17	20
								22
								23

Gantt Chart

Label	P1	P2	P3	P1	P2	P3
state	0	2	7	12	17	18
						21
						23

Multilevel Queue Scheduling Algorithm
(System and User Process)

include <stdio.h>

include <stdlib.h>

include <string.h>

define MAX 100

```
typedef struct {
    int process_id; int type;
    int arrival_time;
    int burst_time;
    int remaining_time;
    int completion_time;
    int response_time;
    int first_response;
    char type[10];
} Process;
```

```
typedef struct {
    Process queue[MAX];
    int front, rear;
} Queue;
```

```
void initQueue (Queue *q) {
    q->front = q->rear = -1;
}
```

```

int isEmpty (Queue *q) {
    return q->front == -1;
}

```

```

void enqueue (Queue *q, process p) {
    if (q->rear == MAX-1) return;
    if (q->front == -1) q->front = 0;
    q->queue[q->rear] = p;
}

```

```

process dequeue (Queue *q) {
    process p = q->queue[q->front];
    if (q->front == q->rear) {
        q->front = q->rear = -1;
    }
    else {
        q->front++;
    }
    return p;
}

```

```

int main() {
    int n; process p[MAX]; Queue sysQ, userQ;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    init(&sysQ); init(&userQ);
    process process-list[MAX];
    for (int i = 0; i < n; i++) {
        printf("Process %d\n", i);
        printf("Enter Arrival time: ");
        scanf("%d", &process-list[i].arrival_time);
        printf("Enter Burst time: ");
        scanf("%d", &process-list[i].burst_time);
        printf("Enter type (0 = system, 1 = user): ");
    }
}

```

```

int t;
scanf("%d", &t); // &process-list[0].type;
if (t == 0) strcpy(process-list[0].type, "system");
else strcpy(process-list[0].type, "user");

```

```

process-list[i].process-id = i;
process-list[i].remaining-time = process-list[i].burst_time;

```

```

process-list[i].completion-time = 0;
process-list[i].response-time = -1;
process-list[i].first-response = 0;

```

```

for (int i = 0; i < n; i++) {
    for (int j = i+1; j < n; j++) {
        if (process-list[i].arrival_time > process-list[j].arrival_time)
            swap(i, j);
    }
}

```

```

process-list[i].process-id = i;
process-list[i].remaining = process-list[i].burst_time;

```

```

int time = 0, completed = 0, i = 0;
int current = -1;

```

```

while (completed < n) {

```

```

    while (i < n && process-list[i].arrival <= time) {
        if (process-list[i].type == 0)
            enqueue(&sysQ, i);
        else
            enqueue(&userQ, i);
        i++;
    }
}

```

```

if (current != -1 && p[current].type == 1 &&
    !isEmpty(&sysQ)) {
    enqueue(&userQ, current);
    current = -1;
}

```

```

if (current == -1) {
    if (!isEmpty(&sysQ))
        current = dequeue(&sysQ);
    else if (!isEmpty(&userQ))
        current = dequeue(&userQ);
    else {
        time++;
        continue;
    }
}

```

```

p[current].remaining--;
time++;

```

```

float totalWT = 0, totalTAT = 0;

```

```

printf("n \ ProcessID \t Type \t AT \t BT \t CT \t WT \t TAT \n");

```

```

for (int i = 0; i < n; i++) {
    p[i].turnaround = p[i].completion - p[i].arrival;
    p[i].waiting = p[i].turnaround - p[i].burst;
}

```

```

totalWT += p[i].waiting;
totalTAT += p[i].turnaround;

```

```

printf("%d \t %s \t %d \t %d \t %d \t %d \t %d \t %d \n",

```

```

    p[i].process-id,
    p[i].type == 0 ? "System" : "User",
    p[i].arrival,
    p[i].burst,
    p[i].completion,
    p[i].waiting,
    p[i].turnaround);
}

```

```

printf("n Average Waiting Time: %.2f", totalWT/n);
printf("n Average Turnaround Time: %.2f", totalTAT/n);

```

```

return 0;
}

```

Output >>

Enter no. of processes: 4

Process 0

Enter Arrival time: 0

Enter Burst time: 5

Enter type (0=System, 1=User): 0

Process 1

Enter Arrival time: 2

Enter Burst time: 3

Enter type (0=system, 1=User)=1

Process 2

Enter Arrival time: 1

Enter Burst time: 4

Enter type (0=system, 1=User)=0

Process 3

Enter Arrival time: 3

Enter Burst time: 2

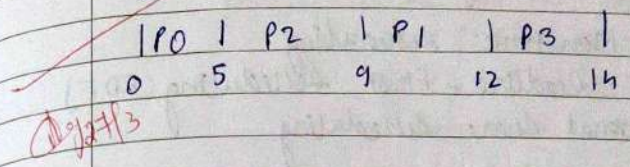
Enter type (0=system, 1=User): 1

Process ID	Type	AT	BT	CT	WT	TAT
0	System	0	5	5	0	5
1	User	2	3	12	7	10
2	System	1	4	9	4	8
3	User	3	2	14	9	11

Average waiting time = 5.00

Average Turnaround time = 8.50

Gantt Chart



RTOS Scheduling Algorithms

- (1) Rate Monotonic Scheduling
- (2) Earliest - Deadline - First Scheduling (EDF)
- (3) Proportional Share Scheduling

#include <stdio.h>

#include <math.h>

#define MAX 10

typedef struct {

int id, burst, deadline, period, completion,
waiting, turnaround; } process;

process p[MAX], temp[MAX];

int n;

float calculate_utilization() {

float u = 0;

for (int i = 0; i < n; i++) {

u += (float)p[i].burst / p[i].period;

}

return u;

}

void sort_deadline() {

for (int i = 0; i < n-1; i++) {

for (int j = i+1; j < n; j++) {

if (p[i].deadline > p[j].deadline) {

process t = p[i];

p[i] = p[j];

p[j] = t;

}

}

}

}

void sort_period() {

for (int i = 0; i < n-1; i++) {

for (int j = i+1; j < n; j++) {

if (p[i].period > p[j].period) {

process t = p[i];

p[i] = p[j];

p[j] = t;

}

}

}

}

void calculate_times() {

int time = 0;

for (int i = 0; i < n; i++) {

p[i].completion = time + p[i].burst;

p[i].turnaround = p[i].completion;

p[i].waiting = p[i].turnaround -

p[i].burst;

time = p[i].completion;

}

}

```
void read() {
    for (int i=0; i<N; i++) {
        p[i] = temp[i];
    }
}
```

int main() {

```
    printf("Enter number of processes: ");
    scanf("%d", &n);
```

```
    for (int i=0; i<n; i++) {
        p[i].id = i;
```

```
        printf("\n Process %d: \n", i);
        printf("Burst Time: ");
        scanf("%d", &p[i].burst);
        printf("Deadline (ten EOF): ");
        scanf("%d", &p[i].deadline);
        printf("Period (ten RMS): ");
        scanf("%d", &p[i].period);
    }
    temp[i] = p[i];
}
```

```
    printf("\n == Earliest Deadline First (EDF)
    scheduling == \n");
```

```
    while = calculate - utilization();
    printf("CPU Utilization: %.2f \n", whl);
```

```
    if (whl <= 1)
        printf("Schedulable Utilization <= 1) \n");
    else
        printf("Not schedulable \n");
```

sort deadline;
calculate_time();

```
    printf("\n EO BT Deadline RT WT TAT \n");
    for (int i=0; i<n; i++) {
        printf("%d %d %d %d %d %d %d \n",
            p[i].id, p[i].burst, p[i].deadline,
            p[i].completion, p[i].waiting, p[i].time);
    }
    3.
```

read();

```
    printf("\n Rate Monitors Scheduling Cons \n");
```

whl = calculate - utilization();

```
    printf("CPU Utilization: %.2f \n", whl);
```

```
    least 3m.bound = n * (p[0].rt, p[0].wt) / (n - 1);
    printf("RM Bound: %.2f \n", rm_bound);
```

```

if (util <= rm_bound)
    printf("Schedulable (Utilization <= RM Bound)\n");
else
    printf("Not Guaranteed Schedulable\n");

// sort by sort period ();
// calculate times ();

printf("ID BT Period CT WT TAT\n");
for (int i=0; i<n; i++) {
    printf("%d %d %d %d %d %d\n",
        p[i].id, p[i].burst,
        p[i].period, p[i].completion, p[i].waiting,
        p[i].turnaround);
}

return 0;
}

```

output >>

Enter number of processes: 3

Process 0:

Burst time: 2

Deadline (for EDF): 4

Period (for RMS): 5

Process 1:

Burst time: 2

Deadline: 4

Period: 5

Process 2:

Burst time: 2

Deadline (EDF): 8

Period (RMS): 10

EDF Scheduling

CPU Utilization: 0.75

Schedulable (Utilization <= 1)

ID	BT	Deadline	CT	WT	TAT
1	2	4	2	0	2
0	3	7	5	2	5
2	2	8	7	5	7

RMS Scheduling

CPU Utilization: 0.75

RM Bound: 0.7798

Schedulable (Utilization <= RM Bound)

ID	BT	Period	CT	WT	TAT
1	2	5	2	0	2
2	2	10	4	2	4
0	3	20	7	4	7

Proportional Scheduling

#include <stdio.h>

#define MAX 10

typedef struct

{ int id, bt, weight;

int ct, wt, rat, share;

}; process;

void proportionalScheduling(process p[], int n){

int totalWeight = 0;

int totalBT = 0;

for (int i=0; i<n; i++){

totalWeight += p[i].weight;

totalBT += p[i].bt;

}

printf("Proportional Scheduling \n");

printf("Total Burst Time: %d \n", totalBT);

printf("Total Weight: %d \n", totalWeight);

int time = 0;

for (int i=0; i<n; i++){

p[i].share = (float) p[i].weight /
totalWeight * totalBT;

time += p[i].share;

p[i].ct = time;

p[i].rat = p[i].ct;

p[i].wt = p[i].rat - p[i].bt;

printf("ID %d | CT %d | Weight %d | Share %d |
BT %d | WT %d | TAT %d \n",
p[i].id, p[i].bt, p[i].weight, p[i].share,
p[i].ct, p[i].wt, p[i].rat);

}

int main() {

process p[MAX];

int n;

printf("Enter number of processes: ");

scanf("%d", &n);

printf("Enter process details: \n");

for (int i=0; i<n; i++){

p[i].id = i;

printf("Process %d: \n", i);

printf("Burst Time: ");

scanf("%d", &p[i].bt);

printf("Weight: ");

scanf("%d", &p[i].weight);

}

proportionalScheduling(p, n);

return 0;

output>>

Enter number of processes: 3

Enter process details:

Process 0

Burst time: 2

Weight: 10

Process 1

Burst time: 4

Weight: 15

Process 2

Burst time: 6

Weight: 20

Proportional scheduling

Total Burst time: 12

Total Weight: 45

ID	0	BT	2	Weight	10	Share	2	CT	2	WT	0	TAT	1
ID	1	BT	4	Weight	15	Share	4	CT	6	WT	2	TAT	6
ID	2	BT	6	Weight	20	Share	5	CT	11	WT	5	TAT	11

MyS1514

Bounded Buffer
(Producer - Consumer Problem)

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <pthread.h>
```

```
#include <semaphore.h>
```

```
#include <unistd.h>
```

```
#define BUFFER_SIZE 5
```

```
#define MAX_ITEMS 10
```

```
int buffer[BUFFER_SIZE];
```

```
int in = 0, out = 0;
```

```
int count = 0;
```

```
sem_t empty, full;
```

```
pthread_mutex_t mutex;
```

```
void *producer(void *arg) {
```

```
    for (int item = 0; item < MAX_ITEMS; item++)
```

```
        sem_wait(&empty);
```

```
        pthread_mutex_lock(&mutex);
```

```
        buffer[in] = item;
```

```
        printf("Produced: %d at buffer[%d]\n",
```

```
               item, in);
```

```
        in = (in + 1) % BUFFER_SIZE;
```

```
        pthread_mutex_unlock(&mutex);
```

```
        sem_post(&full);
```

```
    return NULL;
```

Date: / /
Page:

```

void consumer (void *arg) {
    for (int i=0; i<MAX_ITEMS; i++) {
        sem_wait (&full);
        pthread_mutex_lock (&mutex);

        int item = buffer[out];
        printf ("Consumed: %d from\n", item, out);

        out = (out + 1) % BUFFER_SIZE;

        pthread_mutex_unlock (&mutex);
        sem_post (&empty);
    }
    return NULL;
}

```

```

int main () {
    pthread_t p, c;

    sem_init (&empty, 0, BUFFER_SIZE);
    sem_init (&full, 0, 0);
    pthread_mutex_init (&mutex, NULL);

    pthread_create (&p, NULL, producer, NULL);
    pthread_create (&c, NULL, consumer, NULL);

    pthread_join (p, NULL);
    pthread_join (c, NULL);

    printf ("Execution completed successfully\n");
    return 0;
}

```

Date: / /
Page:

output >>

```

Produced: 0 at buffer [0]
Produced: 1 at buffer [1]
Produced: 2 at buffer [2]
Produced: 3 at buffer [3]
Consumed: 0 from buffer [0]
Consumed: 1 from buffer [1]
Consumed: 2 from buffer [2]
Consumed: 3 from buffer [3]
Produced: 4 at buffer [4]
Produced: 5 at buffer [0]
Produced: 6 at buffer [1]
Produced: 7 at buffer [2]
Produced: 8 at buffer [3]
Consumed: 4 from buffer [4]
Consumed: 5 from buffer [0]
Consumed: 6 from buffer [1]
Consumed: 7 from buffer [2]
Consumed: 8 from buffer [3]
Produced: 9 at buffer [4]
Consumed: 9 at buffer [4]

```

filled
tracing: ~~Produced~~ 0-3 into buffer
Producer

in out Empty Then consumer consumed 0-3
from buffer index (0-3)

Now, the buffer is empty;
The remaining slots are filled with
(in+1) % BUFFER_SIZE, as a
result 4-8 are produced into
buffer

consumer then consumes the ~~buffer~~
index in the buffer

The remaining item is filled and consumed

Dining Philosopher Problem

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
```

```
#define N 5
```

```
sem_t forks[N];
```

```
void *philosopher (void *num) {
    int id = *(int *)num;
```

```
    for (int i=0; i<1; i++) {
        printf("Philosopher %d is thinking\n", id);
        sleep(1);
```

```
        if (id % 2 == 0) {
            sem_wait (&forks[id]);
            printf("Philosopher %d picked up\n left fork %d\n", id, id);
```

```
sem_wait (&forks[(id+1)%N]);
printf
printf("Philosopher %d is eating\n", id);
signal (&forks[id]);
signal (&forks[(id+1)%N]);
signal (&mutex);
}
}
```

```
int main () {
```

```
    pthread_t ph[N];
    int ids[N];
```

```
    sem_init (&mutex, 0, 1);
    for (int j=0; j<N; j++) {
        sem_init (&forks[j], 0, 1);
```

```
        for (int i=0; i<N; i++) {
            ids[i] = i;
            pthread = create (&ph[i], NULL, philosopher,
                               &ids[i]);
        }
```

```
        for (int i=0; i<n; i++) {
            pthread_join (ph[i], NULL);
            return 0;
        }
```

output >>

0 is thinking
 1 is thinking
 2 is thinking
 3 is thinking
 4 is thinking
 3 is eating
 3 is thinking
 4 is eating
 4 is thinking
 2 is eating
 1 is eating

Philosopher	state	Position
P ₀	think	-
P ₁	think	-
P ₂	think	-
P ₃	think	-
P ₄	think	-
P ₃	eat	F ₃ , F ₄
P ₃	think	released F ₃ , F ₄
P ₄	eat	F ₄ , F ₀
P ₄	think	Release F ₄ , F ₀

LAB-6

Banker's Algorithm (Deadlock Avoidance)

#include <stdio.h>

```
int main() {
    int n, m;
    printf("Enter no. of processes: ");
    scanf("%d", &n);

    printf("Enter no. of resource type:");
    scanf("%d", &m);
```

```
int allocation[n][m];
int max[n][m];
int need[n][m];
int available[m];
```

```
printf("\n Enter Allocation Matrix: \n");
for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++) {
        scanf("%d", &allocation[i][j]);
    }
}
```

```
printf("\n Enter Max Matrix: \n");
for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++) {
        scanf("%d", &max[i][j]);
    }
}
```

Date: / /
Page: /

```

printf("In Enter Available Resources: \n");
for (int i=0; i<m; i++) {
    scanf("%d", &available[i]);
}

```

```

for (int i=0; i<n; i++) {
    for (int j=0; j<m; j++) {
        need[i][j] = max[i][j] - allocation[i][j];
    }
}

```

```

int finish[n];
int safeSequence[n];
int work[m];

```

```

for (int i=0; i<n; i++) {
    finish[i] = 0;
}
for (int i=0; i<m; i++) {
    work[i] = available[i];
}

```

```

int count = 0;

```

```

while (count < n) {
    int found = 0;

```

```

    for (int i=0; i<n; i++) {

```

Date: / /
Page: /

```

if (finish[i] == 0) {

```

```

    int possible = 1;

```

```

    for (int j=0; j<m; j++) {
        if (need[i][j] > work[j]) {
            possible = 0;
            break;
        }
    }

```

```

    if (possible) {

```

```

        for (int j=0; j<m; j++) {
            work[j] += allocation[i][j];
        }

```

```

        safeSequence[count] = i;
        count++;

```

```

        finish[i] = 1;
        found = 1;
    }
}

```

```

if (found == 0) {

```

```

    printf("In System is Not in a safe state\n");
    return 0;
}

```

```

printf("System is in safe state\n");
printf("Safe state sequence:");

for (int i = 0; i < n; i++) {
    printf("P%d", safeSequence[i]);
}

printf("\n");
return 0;
}

```

output >> enter number of processes: 5
 enter number of resources: 3
 enter Allocation Matrix:

0	1	0
2	0	0
3	0	2
2	1	1
0	0	2

enter Maximum Demand Matrix:

7	5	3
3	2	2
9	0	2
2	2	2
4	3	3

enter Available Resources:

3	3	2
---	---	---

System is in safe state
 safe sequence is: P1 P3 P4 P0 P2

Deadlock Detection Algorithm

#include <stdio.h>

```

int main() {
    int n, m, i, j, k;

    printf("Enter no. of processes: ");
    scanf("%d", &n);

    printf("Enter no. of resources: ");
    scanf("%d", &m);

    int alloc[n][m], request[n][m];
    int avail[m];
    int finish[n];

    printf("\n Enter allocation matrix: \n");
    for (i = 0; i < n; i++) {
        for (j = 0; j < m; j++) {
            scanf("%d", &alloc[i][j]);
        }
    }

    printf("\n Enter request matrix: \n");
    for (i = 0; i < n; i++) {
        for (j = 0; j < m; j++) {
            scanf("%d", &request[i][j]);
        }
    }
}

```

```

printf("\n Enter Available Resources: \n");
for (i=0; i<m; i++) {
    scanf("%d", &avail[i]);
}

```

```

for (i=0; i<n; i++) {
    finish[i] = 0;
}

```

```

for (j=0; j<m; j++) {
    if (alloc[i][j] != 0) {
        finish[i] = 0;
        break;
    }
}

```

if found;

```

do {
    found = 0;
}

```

```

for (i=0; i<n; i++) {
    if (finish[i] == 0) {

```

```

        for (j=0; j<m; j++) {
            if (highest p[j] > avail[j])
                break;
        }

```

```

    if (j == m) {
        for (k=0; k<m; k++) {
            avail[k] += alloc[i][k];
        }
    }
}

```

```

        finish[i] = 1;
        found = 1;
    }
}

```

```

while (found);

```

```

if deadlock = 0;

```

```

for (i=0; i<n; i++) {
    if (finish[i] == 0) {
        deadlock = 1;
        printf("Process %d is in deadlock.", i);
    }
}

```

```

if (deadlock == 0)

```

```

    printf("\n No deadlock detected \n");

```

```

    return 0;
}

```

out>>

Enter no. of processes: 5

Enter no. of resources: 3

Enter Allocation Matrix:

0 1 0

2 0 0

3 0 3

2 1 1

0 0 2

Enter Request Matrix:

0 0 0

2 0 2

0 0 0

1 0 1

0 0 2

Enter Available Resources:

0 0 0

Process in Deadlock:

No deadlocks detected. All processes can finish.

Q. 8/15/16

Memory Allocation Algorithm for
(1) First Fit (2) Best Fit (3) Worst Fit

#include <stdio.h>

void firstFit (int b[], int m, int p[], int n) {

int u[10] = {0}, a[10];

for (int i=0; i<n; i++) {

a[i] = -1;

for (int j=0; j<m; j++) {

if (!u[j] && b[j] >= p[i])

{

a[i] = j;

u[j] = 1;

break;

}

}

}

printf("\n First Fit\n PNo \t PSize \t Block \n");

for (int i=0; i<n; i++)

if (a[i] != -1)

printf("%d \t %d \t %d \n", i+1, p[i], a[i]+1);

else

printf("%d \t %d \t Not Allocated \n", i+1, p[i]);

}

```
void BestFit(int b[], int m, int p[], int n){
```

```
    int u[10] = {0}, a[10], best;
```

```
    for (int i=0; i<n; i++){
```

```
        a[i] = -1;
```

```
        best = -1;
```

```
        for (int j=0; j<m; j++){
```

```
            {
```

```
                if (!u[j] && b[j] >= p[i])
```

```
                {
```

```
                    if (best == -1 || b[j] < b[best])
```

```
                        best = j;
```

```
                }
```

```
            }
```

```
        if (best != -1)
```

```
        {
```

```
            a[i] = best;
```

```
            u[best] = 1;
```

```
        }
```

```
    }
```

```
    printf("\n Best Fit \n PNo \t PSize \t Block \n");
```

```
    for (int i=0; i<n; i++){
```

```
        if (a[i] != -1)
```

```
            printf("%d \t %d \t %d \n", i+1,  
                p[i], a[i] + 1);
```

```
        else
```

```
            printf("%d \t %d \t Not Allocated \n",  
                i+1, p[i]);
```

```
    }
```

```
void worstFit(int b[], int m, int p[], int n)
```

```
{
```

```
    int u[10] = {0}, a[10], worst;
```

```
    for (int i=0; i<n; i++){
```

```
        a[i] = -1;
```

```
        worst = -1;
```

```
        for (int j=0; j<m; j++){
```

```
            {
```

```
                if (!u[j] && b[j] >= p[i])
```

```
                {
```

```
                    if (worst == -1 || b[j] > b[worst])
```

```
                        worst = j;
```

```
                }
```

```
            }
```

```
        if (worst != -1)
```

```
        {
```

```
            a[i] = worst;
```

```
            u[worst] = 1;
```

```
        }
```

```
    }
```

```
    printf("\n Worst Fit \n PNo \t PSize \t Block \n");
```

```
    for (int i=0; i<n; i++){
```

```
        if (a[i] != -1)
```

```
            printf("%d \t %d \t %d \n", i+1, p[i],  
                a[i] + 1);
```

```
        else
```

```
            printf("%d \t %d \t Not Allocated \n", i+1, p[i]);
```

```
    }
```

W main()

{

```
int m, n, b[10], p[10];
printf("Enter no. of blocks: ");
scanf("%d", &m);
```

```
printf("Enter block sizes: \n");
for (int i=0; i<m; i++)
    scanf("%d", &b[i]);
```

```
printf("Enter no. of processes: ");
scanf("%d", &n);
printf("Enter process sizes: \n");
for (int i=0; i<n; i++)
    scanf("%d", &p[i]);
```

first Fit (b, m, p, n);

best Fit (b, m, p, n);

worst Fit (b, m, p, n);

return 0;

}

out>> Enter no. of blocks: 6

Enter block sizes:

300 600 350 200 750 125

Enter no. of processes: 5

Enter process sizes:

115 500 358 200 375

First Fit:

PNo.	Size	Block
1	115	1
2	500	2
3	358	5
4	200	3
5	375	Not Allocated

Best Fit:

PNo.	Size	Block
1	115	6
2	500	2
3	358	5
4	200	4
5	375	Not Allocated

Worst Fit:

PNo.	Size	Block
1	115	5
2	500	2
3	358	Not Allocated
4	200	3
5	375	Not Allocated

18/5

Lab-8

Page Replacement Algorithms (W.A.F.O (W) LRU (W) Optimal)

```
void printFrames (int f[], int n) {
    for (int i=0; i<n; i++) {
        printf ("%d ", f[i]);
    }
    printf ("\n");
}
```

```
void FIFO (int p[], int n, int f) {
    int fr[f], k=0, faults=0;

    for (int i=0; i<f; i++) fr[i] = -1;

    printf ("\n -- FIFO -- \t Page \t Frames \n");

    for (int i=0; i<n; i++) {
        int found = 0;

        for (int j=0; j<f; j++)
            if (fr[j] == p[i]) found = 1;

        if (!found) {
            fr[k] = p[i];
            k = (k+1)%f;
            faults++;
            printf ("%d\t", p[i]);
        }
        printFrames (fr, f);
    }
    printf ("FIFO Faults = %d\n", faults);
}
```

```
void LRU (int p[], int n, int f) {
    int fr[f], t[f] = {0}, faults=0, cnt=0;
    for (int i=0; i<f; i++) fr[i] = -1;

    printf ("\n -- LRU -- \n Page \t Frames \n");
    for (int i=0; i<n; i++) {
        int found = 0; pos = 0;

        for (int j=0; j<f; j++)
            if (fr[j] == p[i]) {
                t[j] = ++cnt;
                found = 1;
            }
            if (t[j] < t[pos]) pos = j;

        if (!found) {
            fr[pos] = p[i];
            t[pos] = ++cnt;
            faults++;
            printf ("%d\t", p[i]);
        }
        printFrames (fr, f);
    }
}
```

```
printf ("LRU Faults = %d\n", faults); }
```

```
void Optimal (int p[], int n, int f) {
    int fr[f], faults=0;
    for (int i=0; i<f; i++) fr[i] = -1;
    printf ("\n -- Optimal -- \n Page \t Frames \n");

    for (int i=0; i<n; i++) {
        int found = 0;

```

```

for (int j=0; j<f; j++)
    if (fr[j] == p[i]) found = 1;

if (found) {
    int pos = -1;
    for (int j=0; j<f; j++)
        int k;
        if (fr[j] == -1) {
            pos = j;
            break;
        }
    }

    for (k=i+1; k<n; k++)
        if (fr[k] == p[k]) break;

    if (k > pos) {
        for = k;
        pos = j;
    }

    fr[pos] = p[i];
    faults++;
    printf("%d\t", p[i]);
    printFrames(fr, f);
}
}

printf("Optimal Faults = %d\n", faults);
}

```

```

int main() {
    int p[50], n, f;
    printf("Enter no. of pages: ");
    scanf("%d", &n);
}

```

```

printf("Enter reference string: \n");
for (int i=0; i<n; i++)
    scanf("%d", &p[i]);

```

```

printf("Enter no. of frames: ");
scanf("%d", &f);
FIFO(p, n, f);
ERU(p, n, f);
Optimal(p, n, f);
return 0;
}

```

output >> Enter number of pages: 17
 Enter reference string:
 7 2 3 1 2 5 3 4 6 7 7 1 0 5 4 6 2
 Enter number of frames: 4

--- FIFO ---

Page	Frames
7	7---
2	72--
3	723-
1	7231
5	5231
4	5431
6	5461
7	5467
1	1467
0	1067
5	1057
4	1054
6	6054
2	6254
FIFO Faults = 14	

--- LRU ---

Page	Frames
7	7 - - -
2	7 2 - -
3	7 2 3 +
1	7 2 3 1
5	5 2 3 1
4	5 2 3 4
6	5 6 3 4
7	7 6 3 4
1	7 6 1 4
0	7 6 1 0
5	7 5 1 0
4	4 5 1 0
6	4 5 6 0
2	4 5 6 2

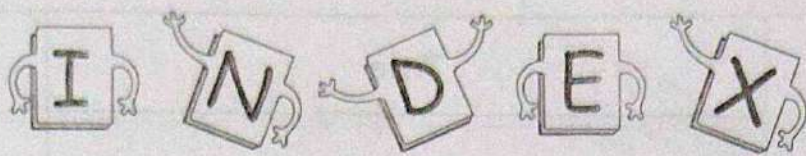
LRU Faults = 14

--- Optimal ---

Page	Frames
7	7 - - -
2	7 2 - -
3	7 2 3 -
1	7 2 3 1
5	7 5 3 1
4	7 5 4 1
6	7 5 6 1
0	0 5 6 1
4	4 5 6 1
2	2 5 6 1

Optimal Faults = 10

22/5/26



Name Praseen Choudhary ^{Sum} 4 ^{Std} 4 Sec

Roll No. Subject School/College

School/College Tel. No. Parents Tel. No.

Sl. No.	Date	Title	Page No.	Teacher Sign / Remarks
1	6/3/26	FCFS		
2	6/3/26	SJF (Non Preemptive)		} dy 13/3/26
3	6/3/26	SJF (Preemptive)		
4	13/3/26	Priority (Preemptive)		
5	13/3/26	Priority (Non Preemptive)		} dy 13/3/26
6	27/3/26	Round Robin		
7	27/3/26	Multi-Level Queue		} dy 27/3
8	10/4/26	Late Monotonic Scheduling		
9	10/4/26	Earliest Deadline First	(10)	} dy 8/5/26
10	10/4/26	Proportional Share Scheduling		
11	24/4/26	Bounded Buffer (Producer-Consumer)		} (10) SK ml/w
12	24/4/26	Dining Philosopher Problem		
13	8/5/26	Deadlock Avoidance		
14	8/5/26	Deadlock Detection	(10)	} dy 8/5/26
15	15/5/26	Memory Allocation (First Fit, Best Fit, Worst Fit)	10	SK 10/5
16	22/5/26	Page Replacement Algorithm (FIFO, LRU, Optimal)		